SOLUTION AND WORKFLOW REPORT

Salesforce QA Agent

A Chrome extension and local AI agent that runs Salesforce test cases as real users, collects evidence, files defects and checks test cases against the requirements

Version	Extension 0.2.0 · Agent server 0.1.0
Date	22 October 2026
Prepared by	Mayank Jain
Org in use	DRIVE-ENERGY-4828 - TMCDEV (sandbox); UAT for the RMS project

At a glance: trial runs while building the agent

Trial runs (agent testing)	Test cases tried	Bugs filed in trials	Management reports
93 (1–21 Oct 2026)	**18** distinct test cases	**10**, all with screenshot evidence	**3**: execution, defects, test case review; per project and org

Contents

1. Executive summary

2. System overview and architecture

3. Components

4. One-time setup workflow

5. Running a single test case

6. How a result is decided

7. Running a batch from Excel / CSV

8. Checking test cases against the BRD

9. Reports

10. Tests with several users and API checks

11. Hybrid execution and Fast execution

12. Projects and org folders

13. Jira story to test cases

14. Sanity and Regression test sets

15. Security and safeguards

16. Operating the agent

17. Change history (6–21 October)

18. Limitations and next steps

Each entry links to its section.

1. Executive summary

The Salesforce QA Agent turns written test cases into executed, evidenced test results without a tester clicking through Salesforce. A tester opens any Salesforce page, picks the org and the persona (the Salesforce user the test should run as), pastes or uploads test cases, and the agent does the rest.

> **Current status (22 October 2026).** The agent is built and has been tried out on the DRIVE-ENERGY-4828 - TMCDEV sandbox (and UAT for the RMS project). The 93 runs of 18 test cases and the 10 bugs recorded so far are trial runs used to build and check the agent itself, not results of a real test cycle; real testing of the org has not started yet. All 197 automated tests pass. Since the first version of this report, Jira story generation, Sanity and Regression test sets with re-checks and run comparison, project folders, multi-persona tests, API checks, hybrid execution and the Fast execution switch have been added (section 17).

What it does

Capability	What happens
Runs tests as real users	Each test logs in as the persona's actual Salesforce user through the admin's **Login As**, in a separate, isolated browser. Nobody types or stores persona passwords.
AI executes the steps	Claude (through Claude Code) reads the test case, performs each step in the browser, verifies the result and records a PASS / FAIL / BLOCKED status for every step.
Evidence for every step	Every step has its own screenshot. A step without a screenshot is rejected, so results can always be audited.
Defects filed automatically	Failures become bug reports with steps to reproduce, expected vs actual, severity and screenshots. Repeat failures are linked to the existing bug instead of being filed twice.
Batches from Excel / CSV	A whole test sheet can be uploaded, checked row by row, and run one test after another, with a batch summary.
Test cases checked against the BRD	Before testing, the BRD or requirement sheet can be uploaded to check that each test case is correct and traceable, and to find requirements no test case covers.
Tests with several users	One test can involve up to 4 personas (for example Sales submits, the Sales Manager approves), each in its own browser.
API checks	Claude can check the org's REST API, or an approved external system, where the UI cannot show the result. Each call is saved as evidence.
Jira story to test cases	A pasted story becomes reviewed test cases (positive, negative, boundary, permission…) with traceability to its acceptance criteria.
Sanity and Regression sets	Named sets run with one click. Failures are re-checked once to separate real failures from flaky ones, and each run is compared with the previous one.
Faster, steadier runs	Hybrid execution does known actions in one call; the Fast execution switch overlaps login with AI start-up. Every run reports its execution time.
Projects and org folders	Reports, bugs, notes, test cases and test sets are kept per project and per org (for example RMS / TMCDEV and RMS / UAT).

Why it matters

- **Less manual effort:** testers write or upload test cases; execution, screenshots and defect write-ups are produced by the agent.
- **Consistent, auditable results:** the same rules decide every verdict, and every step carries evidence.
- **Earlier quality checks:** wrong or missing test cases are caught against the requirements before a test cycle starts.
- **Management visibility:** three plain-language reports show pass rate, open defects by severity and requirement coverage, and print to PDF.

2. System overview and architecture

The solution has three parts that run on the tester's own Windows machine, plus the Salesforce org being tested. The extension is only the front end; all the work happens in the local agent server, which never accepts connections from the network (it listens on `127.0.0.1` only).

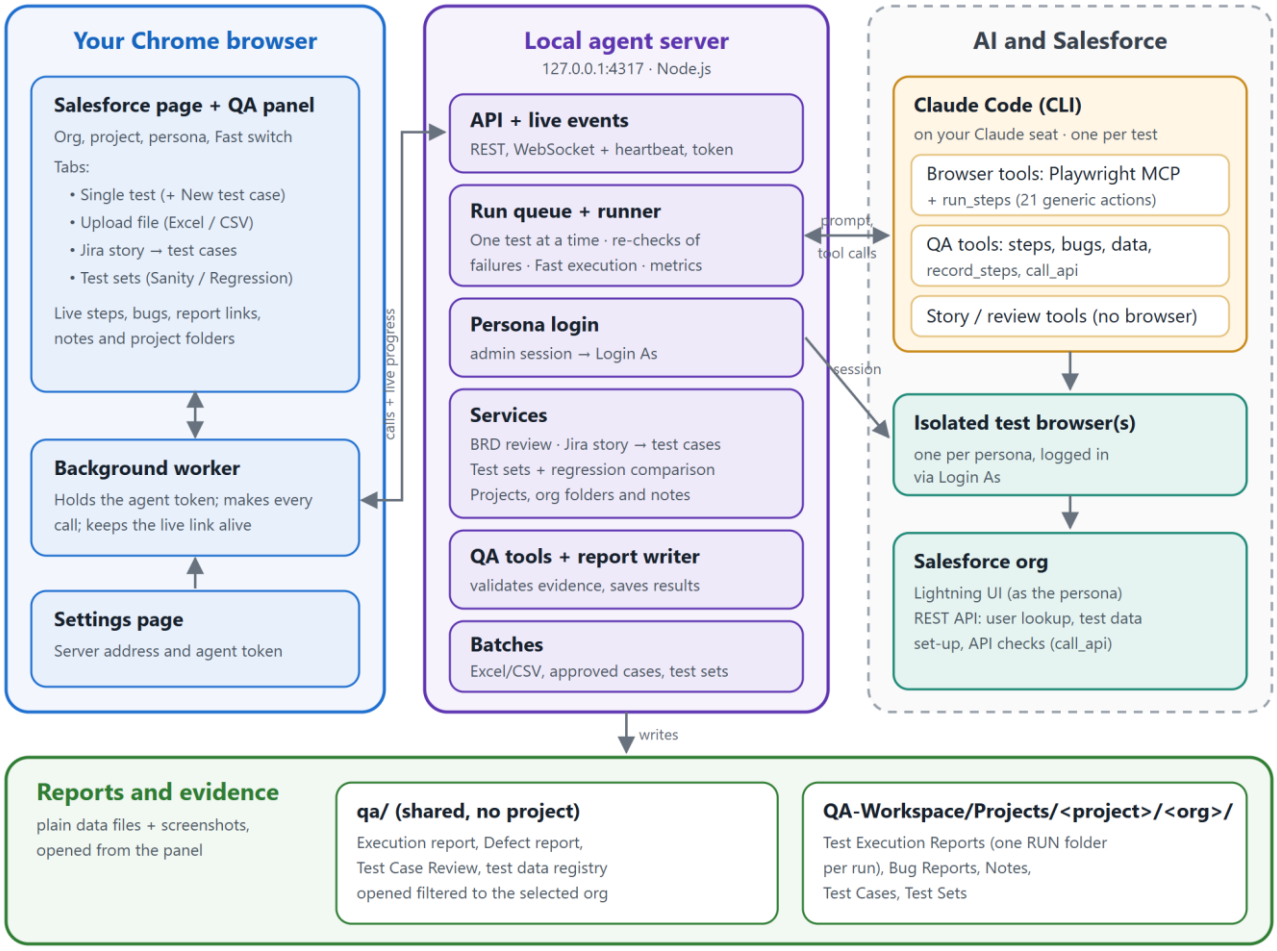


Figure 1. Architecture. The extension talks only to the local server. The server starts Claude Code with the browser tools (Playwright MCP plus run_steps for several known actions in one call) and the QA tools through which Claude reports results. Jira stories and BRD reviews use their own tools without a browser. Results are saved in qa/ or, when a project is selected, in that project's folder for the org.

Colour key used in every diagram: **blue** = Chrome extension, **purple** = local agent server, **amber** = Claude (AI), **teal** = Salesforce, **green** = reports.

Technology

TypeScript throughout, run directly by Node.js 22+; Chrome Manifest V3 extension built with esbuild; Fastify for the server; Playwright for browsers; Claude Code CLI with the Model Context Protocol (MCP) for the AI tools; Zod for validation; read-excel-file, fflate and unpdf to read Excel, Word and PDF files.

Where the data goes

Test cases, screenshots and results stay on the tester's machine in the project folder. The only external services are the Salesforce org under test and Claude (through the tester's Claude Code sign-in), which receives the test case or BRD text and the page snapshots needed to do the work.

3. Components

Component	Responsibility	Where
QA panel (content script)	Floating button and panel injected on Salesforce pages. Detects the org from the page address, manages personas, takes a pasted test case or an uploaded file, shows live steps, bugs and results, and links to the reports.	`packages/extension/src/content/`
Background worker	Holds the agent token and server address and makes every server call, so the Salesforce page never sees the token. Forwards the live event stream of a run to the panel.	`extension/src/background.ts`
Settings page	Where the tester pastes the agent token once and tests the connection.	`extension/src/options.ts`
Agent server	REST and WebSocket API on 127.0.0.1:4317. Registers orgs, saves admin sessions, searches users for personas, queues and runs tests, parses uploaded sheets, runs requirement reviews and serves the reports behind a secret link.	`packages/server/src/`
Runner	Executes one run at a time: persona login, hand-over to Claude, step and bug collection, verdict, report update, and clean-up of the persona session file.	`server/src/runs/runner.ts`
Persona login	Uses the org admin's saved session and Salesforce's Login As to open an isolated browser as the persona's user, and confirms the "Logged in as" banner.	`server/src/salesforce/`
Claude Code + MCP tools	Claude runs headless on the signed-in Claude Code seat (no API key billing). It only has two tool sets for a test: Playwright MCP with run_steps (browser) and the QA tools (record_step, report_bug, finish_test, test-data tools); for reviews, only the review tools.	`server/src/agent/`
Batch parser	Reads .xlsx / .csv test sheets, finds the header row, matches columns and personas loosely and flags invalid rows.	`server/src/suites/`
Review service	Extracts text from the BRD (PDF, Word, Excel, CSV, TXT, MD), runs Claude with the review tools, validates every verdict and saves the review.	`server/src/reviews/`
Reports	Three report pages that read data files in the qa/ folder; status changes made by reviewers can be saved back into the data files.	`qa/reports`, `qa/bugs`, `qa/reviews`
Shared types	Schemas used by both extension and server, so requests are validated identically on both sides.	`packages/shared/`

Component	Responsibility	Where
Browser tools + actions	Playwright MCP plus run_steps: 21 generic actions (navigate, click, fill, select, lookup, waits, getField, verify, screenshot…) with Salesforce-aware waits. Stops and hands back to Claude when unsure.	`server/src/agent/browser-mcp.ts, actions.ts`
API checks	call_api: org REST API as the admin or a persona, and external systems listed in config/apis.json; request and response saved as evidence.	`server/src/agent/api-call.ts`
Run metrics	Per run: total time, login, Claude start-up and report time, turns, tokens, browser actions, batches and fallbacks.	`server/src/agent/metrics.ts`
Project store	Creates projects and org folders in QA-Workspace, notes, and opens the right reports. Validates every name and path.	`server/src/projects/`
Generation service	Jira story to test cases: runs Claude with the story tools, saves cases, review states and Excel exports.	`server/src/generation/`
Test set service	Saves Sanity and Regression sets, runs them as batches, collects final results (after re-checks) and compares runs.	`server/src/testsets/`
Suite store (re-checks)	Tracks batch progress and starts one re-check for each FAIL or BLOCKED case when asked; marks it confirmed or flaky.	`server/src/suites/store.ts`

4. One-time setup workflow

Setup is done once per machine (steps 1–2), once per org (step 4) and once per persona (step 5). After that, a tester only opens Salesforce and runs tests.

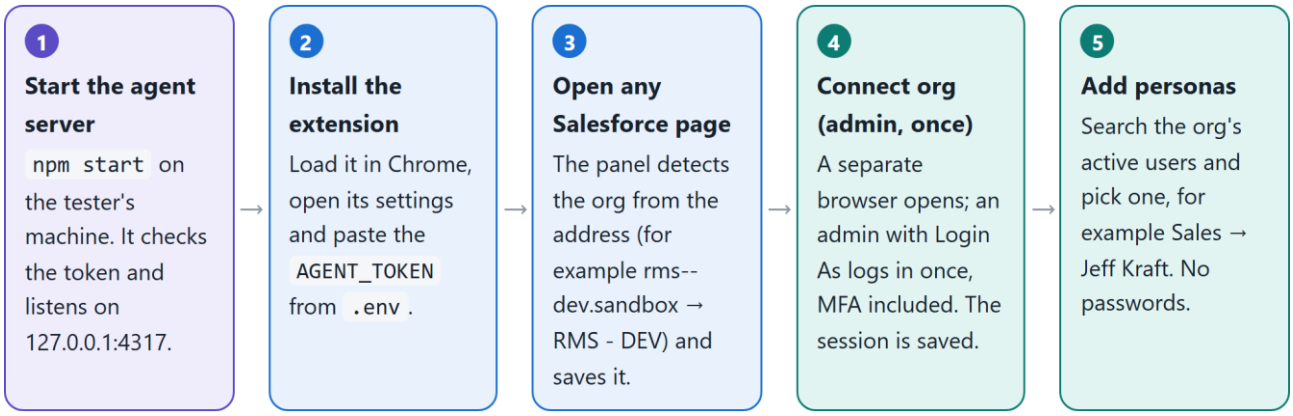


Figure 2. Setup steps. When Salesforce eventually expires the admin session, the panel shows Reconnect; only step 4 is repeated.

1. **Start the agent server:** `npm start` on the tester's machine. It checks the token and listens on 127.0.0.1:4317.
2. **Install the extension:** load it in Chrome, open its settings and paste the `AGENT_TOKEN` from `.env`.
3. **Open any Salesforce page:** the panel detects the org from the address (for example rms--dev.sandbox becomes RMS - DEV) and saves it.
4. **Connect org (admin, once):** a separate browser opens; an admin with Login As logs in once, MFA included. The session is saved.
5. **Add personas:** search the org's active users and pick one, for example Sales → Jeff Kraft. No passwords are needed.

Why personas need no passwords

Personas are real Salesforce users. The agent logs in as them the same way an administrator clicks **Login** on a user record in Setup: it opens Salesforce's Login As address with the admin's saved session, in a brand-new browser that shares nothing with the tester's own Chrome.

Before opening any browser it checks, through the Salesforce API, that the admin session still works and that the persona's user exists and is active, so problems are reported in seconds with a clear fix.

What the panel shows

- Connection to the agent and the detected org
- Admin connection status, with **Connect** / **Reconnect**
- Persona picker; **+ Add** searches the org's users
- **Single test** tab: paste and run, with live steps, screenshots, bugs and log
- **Upload file** tab: batches and the requirements check

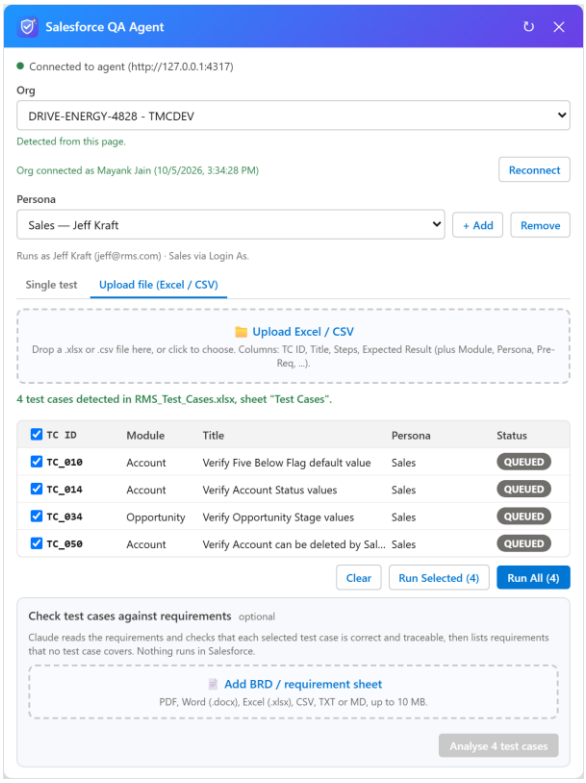


Figure 3. The QA panel (illustrative data) with an uploaded test file and the optional requirements check.

5. Running a single test case

The tester pastes a test case (it can be a row copied from a test sheet), optionally a title, and clicks **Run test**. The sequence below is what happens next. The panel updates live throughout, and the tester can keep working.

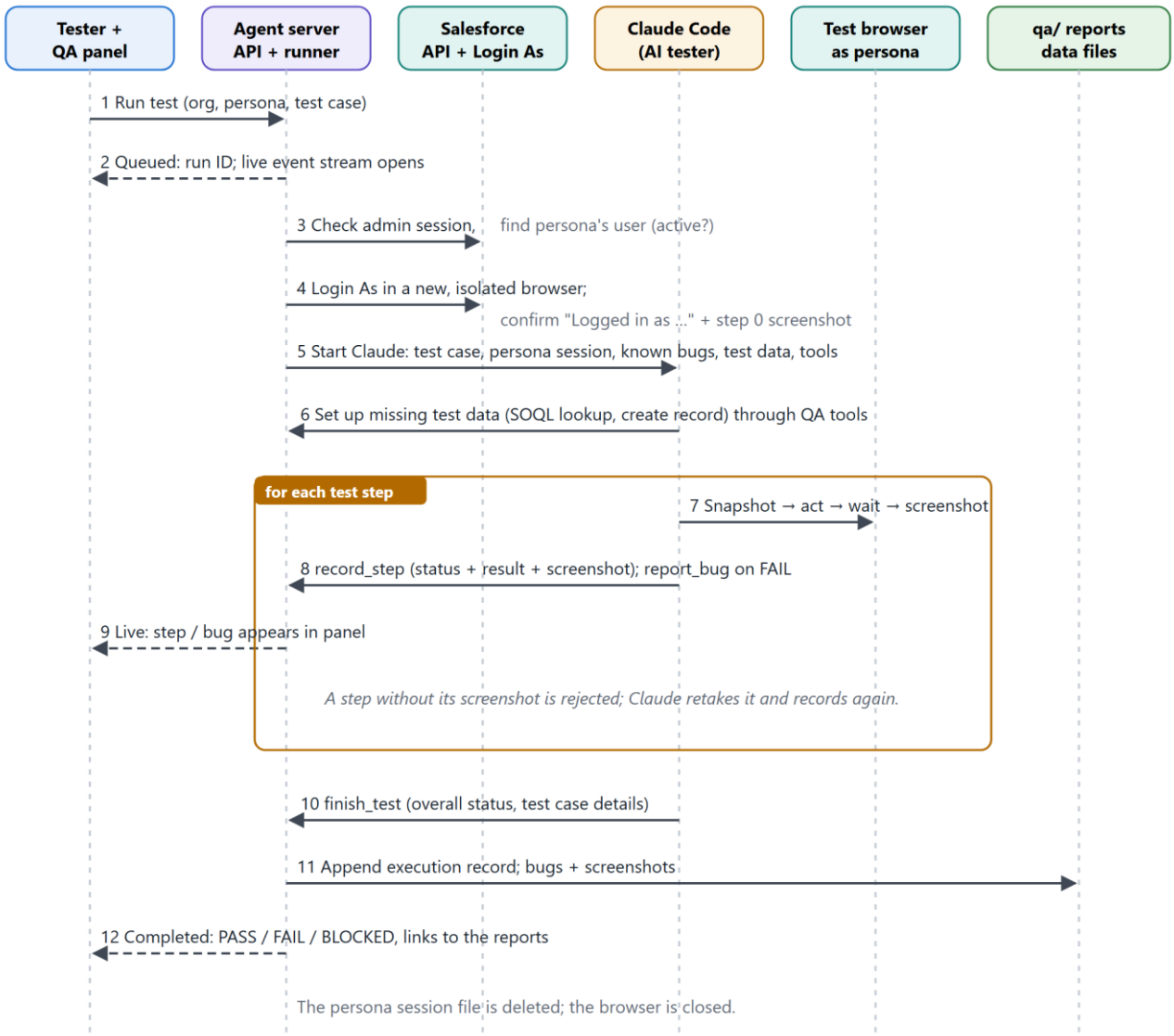


Figure 4. Sequence of one test run. Solid arrows are requests or tool calls; dashed arrows are live updates to the panel. Steps 7–8 repeat for every step of the test case.

Step by step

1. The tester clicks **Run test**; the panel sends the org, persona and test case to the server.
2. The server queues the run and returns a run ID; the panel opens a live event stream.
3. The server checks the admin session and finds the persona's Salesforce user (and that it is active).
4. It logs in as the persona with **Login As** in a new, isolated browser, confirms the "Logged in as …" banner and takes the step 0 screenshot.
5. It starts Claude with the test case, the persona session, known open bugs, earlier test data and the tools.
6. Claude sets up any missing test data (SOQL lookups, record creation) through the QA tools.
7. For each step, Claude reads the page, acts, waits for the result and takes a screenshot.

8. Claude records the step (status, observed result, screenshot) and files a bug on FAIL. A step without its screenshot is rejected and retaken.
9. Each step and bug appears live in the panel.
10. Claude calls **finish_test** with the overall status and the test case details.
11. The server appends the execution record, bugs and screenshots to the reports.
12. The panel shows PASS / FAIL / BLOCKED with links to the reports. The persona session file is deleted and the browser closed.

What Claude is told to do

- Use only the browser tools and the QA tools. Never log out, switch user, change Setup configuration or delete records.
- Read the page before acting, use visible labels, and wait for Lightning spinners and toasts before judging a result.
- Verify thoroughly: list every picklist value seen, confirm saves after refresh, quote error messages exactly.
- Reuse test data created by earlier runs; create only what is missing, with the admin's API access, and only to prepare preconditions (never to perform the action being tested).
- Link a failure to a known open bug instead of filing a duplicate.

Added since 5 October

- **New test case** clears the form (title, test case, result, log) to start the next test without reloading the page.
- **Also involves** adds more personas to a test; see section 10.
- **Fast execution** switch: the run uses the faster path described in section 11. Off runs exactly as before.
- **Project**: with a project selected, the run is saved in that project's folder for the org (section 12).
- **Results survive Chrome suspending the extension**: the panel restores the last run, its steps and its report links when it is reopened.
- The finished run shows its **execution time** and a link to its own entry in the execution report.

6. How a result is decided

Each step gets one status from Claude: **PASS** (behaves as expected), **FAIL** (the application does not behave as the test case expects, including a field that is hidden or read-only for the persona; a bug is filed), **BLOCKED** (cannot be tested for reasons outside the behaviour under test, such as test data that could not be created), or **N/A**. The overall verdict is then decided by fixed rules on the server, the same for the panel and the reports:

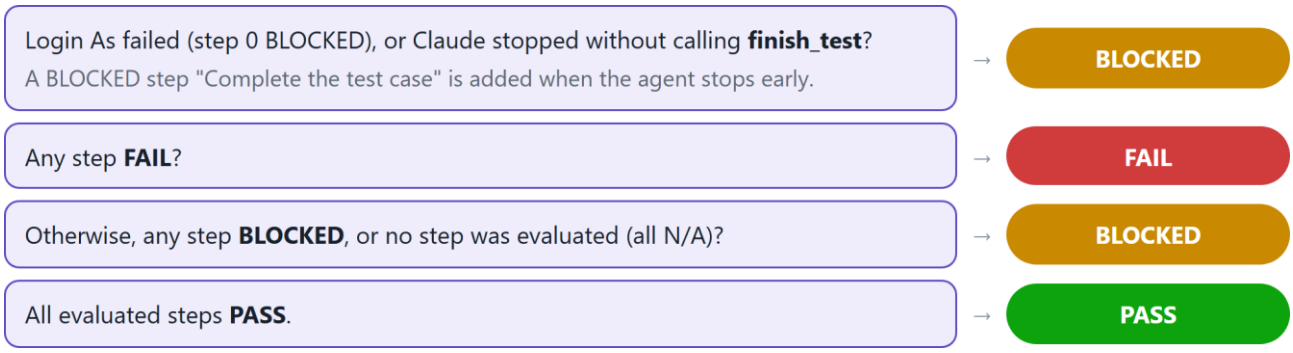


Figure 5. Verdict rules, checked top to bottom. Claude's own overall status can only make the verdict stricter (for example PASS → FAIL), never more lenient.

Evidence and defect rules

Rule	Effect
Every non-N/A step needs a screenshot	The server rejects `record_step` until the named screenshot exists, so every result can be checked.
A bug needs at least one screenshot	Bugs are filed with steps to reproduce, expected vs actual, severity (Critical / High / Medium / Low), priority and the record used; evidence is copied to `qa/bugs/<BUG-ID>/`.
Repeat failures	A failure matching a known open bug is recorded as a new occurrence of that bug (with its run and screenshots) rather than a duplicate.
Possible intentional behaviour	Filed with type "Requirement gap" (for example field-level security), so a reviewer can mark it "Not a Bug".
finish_test only once, only after steps	Calls in the wrong order are rejected with an explanation Claude can act on.

7. Running a batch from Excel / CSV

In the panel's **Upload file (Excel / CSV)** tab, a whole test sheet (up to 5 MB, 200 rows per batch) is turned into individual runs. Nothing runs until the tester has reviewed what was detected.

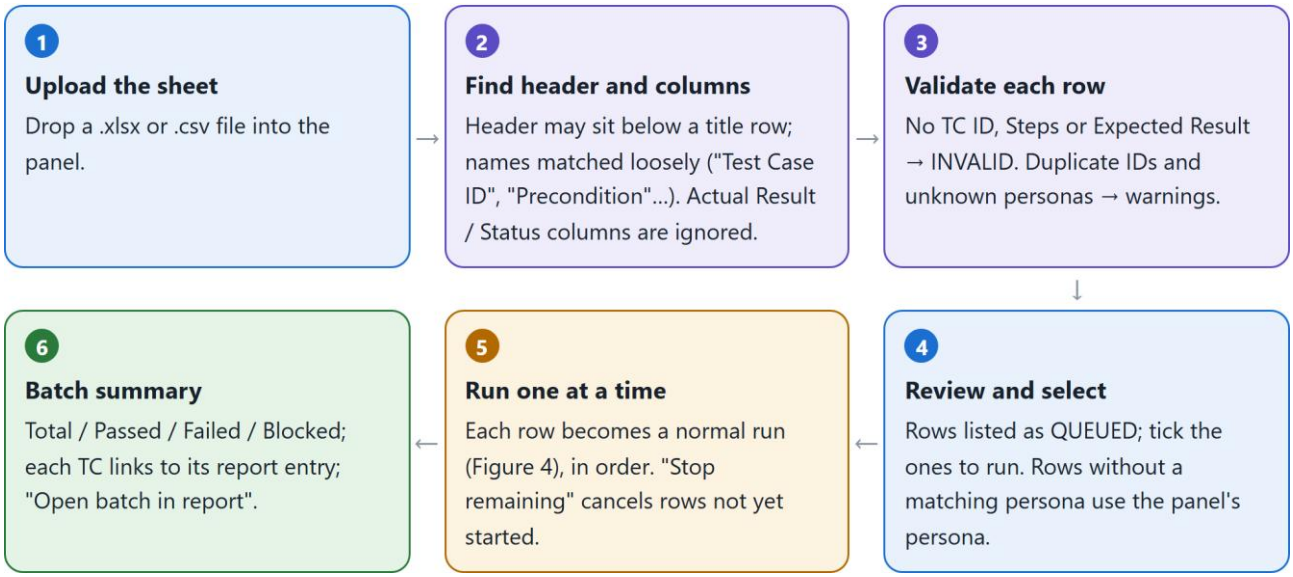


Figure 6. Batch workflow. If a required column is missing, the panel names it and nothing runs. Optionally, before step 5, the same selected rows can be checked against the BRD (section 8).

Columns	Accepted names (examples)
Required	TC ID, Title, Steps, Expected Result
Optional	Sprint, Story, Module, Persona, Description, Pre-Req / Precondition, Type, Priority, Test Data
Ignored	Actual Result, Status, Remarks, Executed By / On (results of an earlier execution are not instructions)

Runs execute strictly one after another, because every run shares the org admin's saved session; parallel browsers would compete for it.

Added since 5 October

- **Re-check failures once** (optional for uploaded files): every FAIL or BLOCKED case runs once more; failing again confirms it, passing marks it **flaky**. The summary shows a Flaky count.
- **Save as test set (N)** keeps the selected rows as a named Sanity or Regression set (section 14).
- A batch uses the selected **project** and the **Fast execution** switch, like a single test.
- A Persona cell may name several personas (`Sales User, Sales Manager`) for multi-user tests.

8. Checking test cases against the BRD

After a test sheet is loaded, the optional **Check test cases against requirements** section accepts the BRD or requirement sheet. Claude reads both and judges every selected test case. Nothing runs in Salesforce and no org connection is needed; a review of a few test cases takes under a minute.

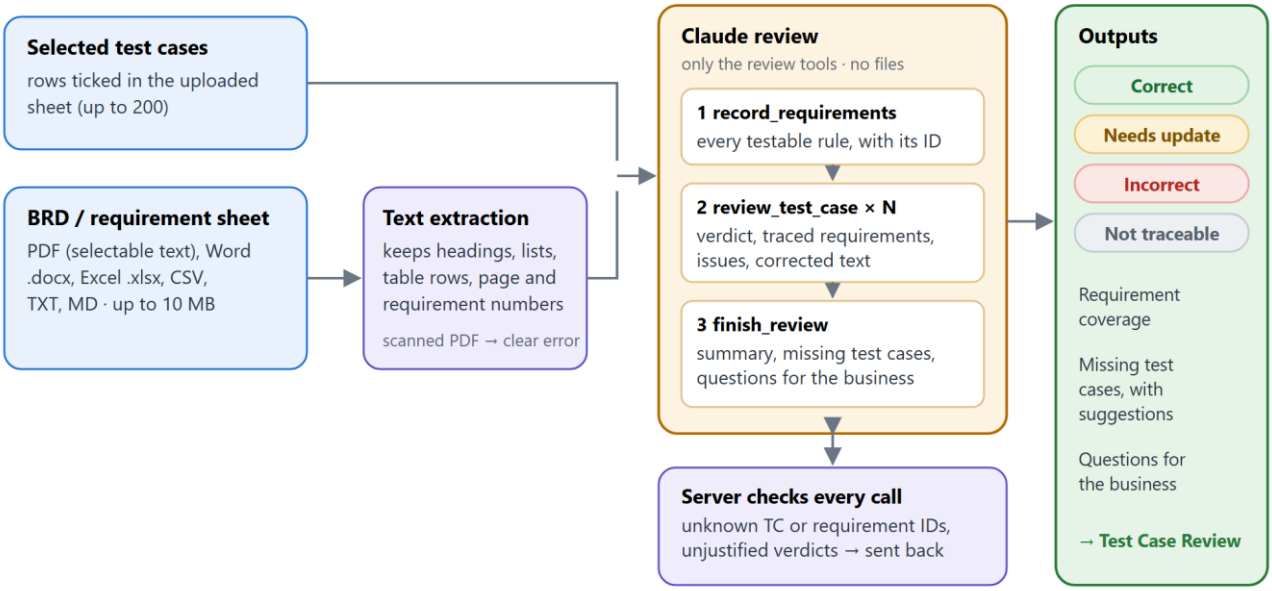


Figure 7. Requirements review. Progress shows live in the panel ("2 of 4 test cases done"). If Claude stops part-way, what was reviewed is still saved and marked "Incomplete".

Verdict	Meaning
Correct	Traceable to a requirement; steps, precondition, persona and expected result match it. Must name the requirement(s).
Needs update	Traceable and mostly right, but something should change (incomplete expected value, missing step, weak precondition, wrong persona). Must list the issues and give corrected text.
Incorrect	Contradicts the requirement, so the test would pass wrong behaviour or fail correct behaviour. Must list the issues and give corrected text.
Not traceable	No requirement in the document covers it.

Accepted documents

PDF (with selectable text), Word .docx, Excel .xlsx, CSV, TXT and MD, up to 10 MB. Headings, list items, table rows and requirement numbers are kept when the text is extracted. A scanned PDF gets a clear message asking for the Word version.

Verified end to end. On a sample BRD with five requirements and four test cases, the review correctly marked TC*010 Incorrect (expects the Five Below Flag to default to True; the BRD says False),* TC014 Needs update (misses the "Prospect" default), TC*034 Correct and* TC050 Not traceable (nothing covers deleting Accounts), and proposed four missing test cases, in about 30 seconds.

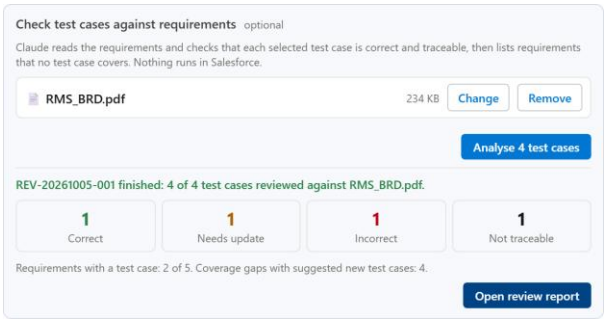


Figure 8. The requirements check in the panel after a review (illustrative data).

9. Reports

All results are kept in plain data files in the `qa/` folder, the source of truth. Three report pages read them and are opened from the panel through a secret link. Each report starts with a one-line verdict and a plain-English summary for management, then key figures, charts and the detail; each can be filtered, exported to CSV and printed to PDF.



Figure 9. Report data flow. Reviewers can change a run or bug status on the page (with a note and their name); the change is kept in the browser until they save it back into the data file.

Report	What a manager sees first	Detail available
Test Execution	Verdict (On track / At risk / Needs attention), pass rate, results bar, open defects to watch, results by module. Counts the latest result of each test case by default; "All runs" includes re-runs.	Per test: expected vs actual, steps, screenshots, status history, status change.
Defect Report	Open defects, Critical / High count, open defects by severity and status, defects by module, average age.	Steps to reproduce, screenshots, every run that hit the bug.
Test Case Review	Verdict (Ready to test / Minor / Significant rework), % correct, requirement coverage, missing test cases.	Issues per test case with current and suggested text side by side; earlier reviews.

The reports as they look today

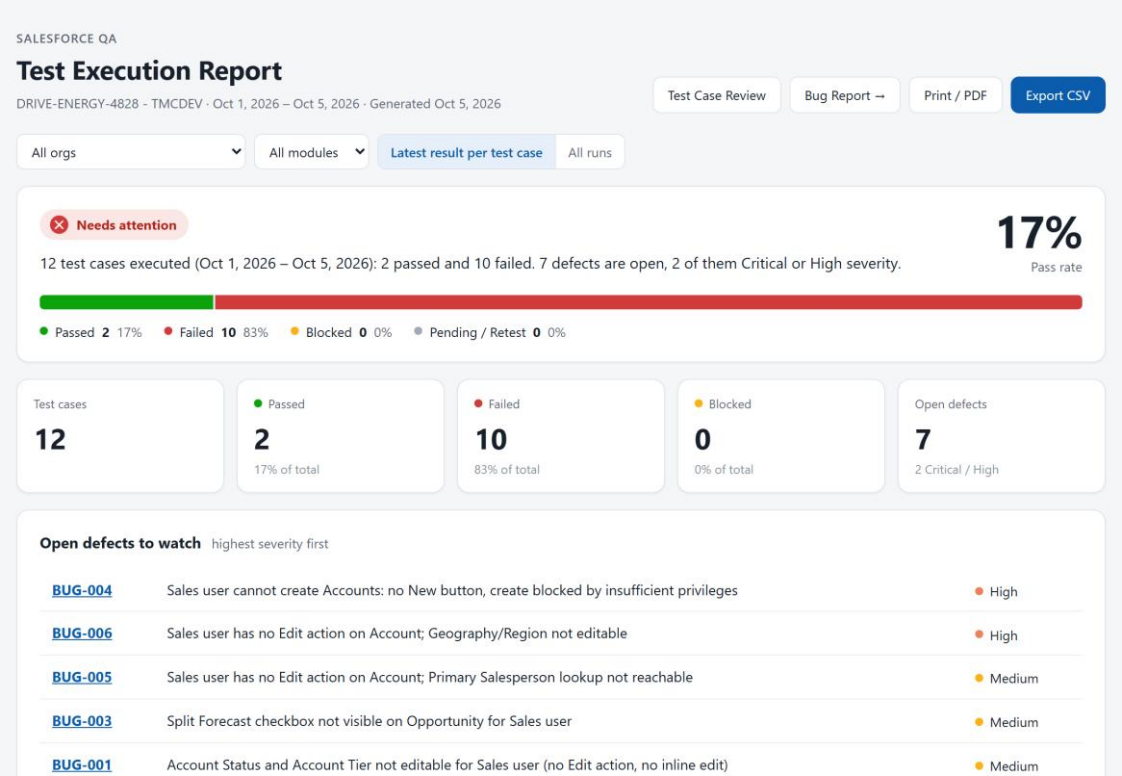


Figure 10. Test Execution Report showing trial runs on the TMCDEV sandbox while the agent was being built (12 test cases, 7 bugs at the time). Not results of a real test cycle.

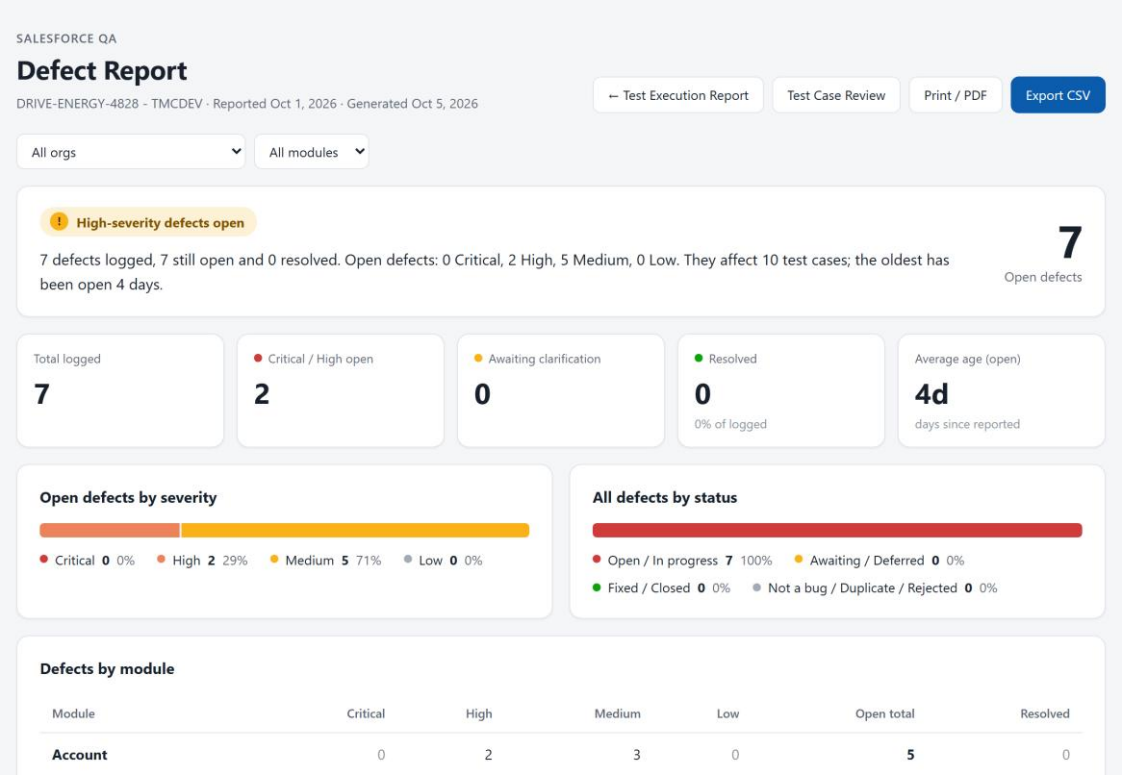


Figure 11. Defect Report showing the bugs filed during trial runs (7 at the time). They show how defects are reported; they are not confirmed defects of the org.

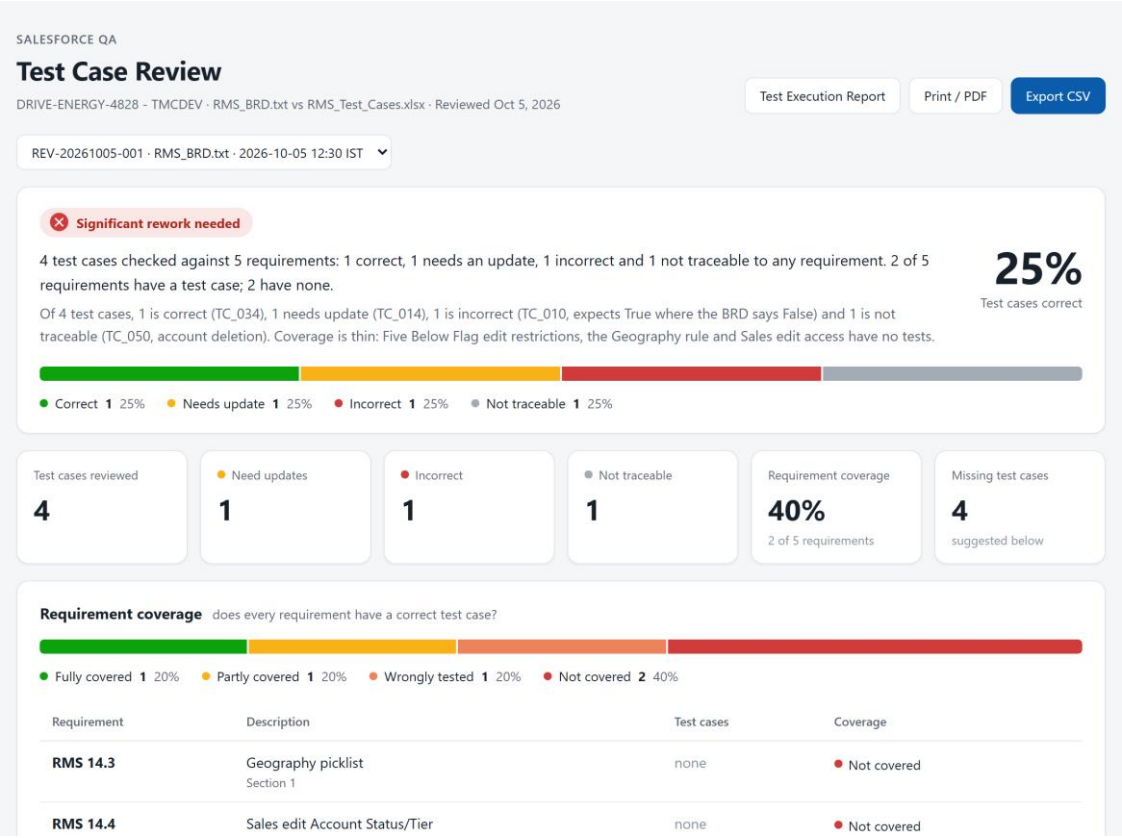


Figure 12. Test Case Review from the end-to-end check on a sample BRD: 1 of 4 test cases correct, 2 of 5 requirements with a test case, 4 missing test cases suggested.

Added since 5 October

- **Execution time**: a Time column per test and an Execution time tile; the CSV export includes durationSec and the execution mode (Standard or Fast).
- **Re-check of**: a re-check run names the run it re-checked.
- **API evidence**: request and response files of API checks are linked from the step that used them.
- **Org filter**: shared reports opened from the panel show only the selected org; older records without an org show as Unassigned.
- **Per project and org**: with a project, each org folder has its own Test Execution and Bug reports, one RUN folder per run, and bug numbers starting at BUG-001.

10. Tests with several users and API checks

Some business flows need more than one user, and some results cannot be seen in the UI. Both are now covered by the same run, with the same evidence rules.

Several users in one test

- **Single test:** pick the first persona, then tick the others under **Also involves**. **Excel / CSV:** list them in the Persona cell, first one first (`Sales User, Sales Manager`; also `+`, `;`, `->`, `then`).
- Every persona logs in through Login As before the test starts and gets **its own browser** for the whole run. Up to 4 personas per test.
- Each step is done in the browser of the persona it is for and is labelled with it, for example `[Sales Manager] Approve the discount`.
- Right after the logins the run checks every session is still active. If Salesforce ended an earlier Login As when a later one started, the run is BLOCKED with that explanation.

API checks (integration testing)

Target	What is allowed
Org REST API	As the admin, or as a persona to check that user's own access. Only GET on /services/data/. POST, PUT and PATCH only to Apex REST endpoints, for example to simulate an external system pushing data in.
External systems	Only base URLs listed in config/apis.json. Read-only unless the entry says allowWrite. Secrets stay in .env and are referenced as ${NAME}, so Claude never sees them.
Evidence	Each request and response is saved as api_NN_….json next to the screenshots, shown in the panel and linked from the report. A step checked through an API uses that file as its evidence.

11. Hybrid execution and Fast execution

Claude still reads the test case, decides every action and judges every result. What changed is how much work each Claude decision does, and how much waiting happens before Claude starts.

Hybrid execution

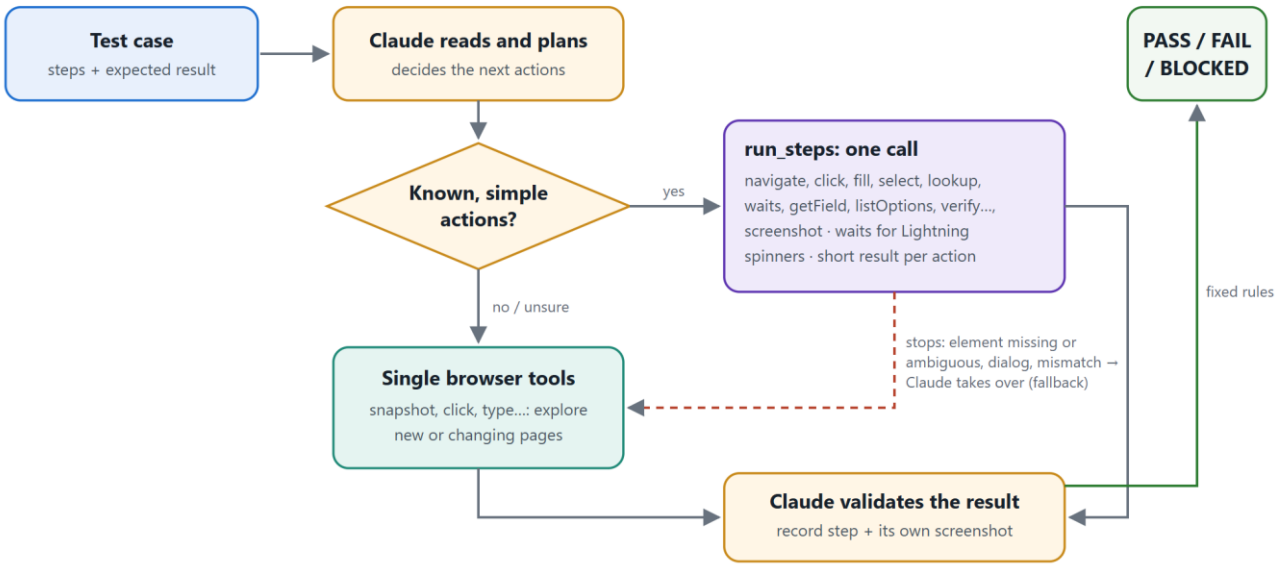


Figure 13. Hybrid execution. Known, simple actions run as one run_steps call; anything new, dynamic or uncertain uses the single browser tools. run_steps stops at the first doubt and Claude takes over. Verdict rules are unchanged.

- **run_steps** holds no business logic. It finds elements by visible label, role or text, waits for Lightning spinners (no fixed waits), and returns one short line per action plus any toasts, errors and dialogs.
- It stops at an element it cannot find for certain, an unexpected dialog, or a check that does not match. Screenshots taken by it are saved as evidence but not sent to Claude, which saves tokens.
- `HYBRID_EXECUTION=false` in .env goes back to one Claude decision per browser action.

Fast execution switch

The switch in the panel applies to the runs started from it (single tests, uploaded files, approved Jira cases, test sets). **Off** runs exactly as before. **On** changes three things:

1. **Claude starts while the persona logs in.** Claude may plan but cannot touch the browser until the login has been confirmed (a login gate). If the login fails, the run is BLOCKED as before.
2. **Several finished steps are recorded in one call** (record_steps). Each step remains its own entry with its own screenshot; merged steps such as "Steps 2-3" are rejected.
3. **Claude is told which records the last run of the same test case used**, so it can open them directly instead of searching.

Validations, screenshots, negative checks and the PASS / FAIL / BLOCKED rules are the same in both modes. `FAST_EXECUTION=true` in .env sets the default for runs that don't say.

Measured results

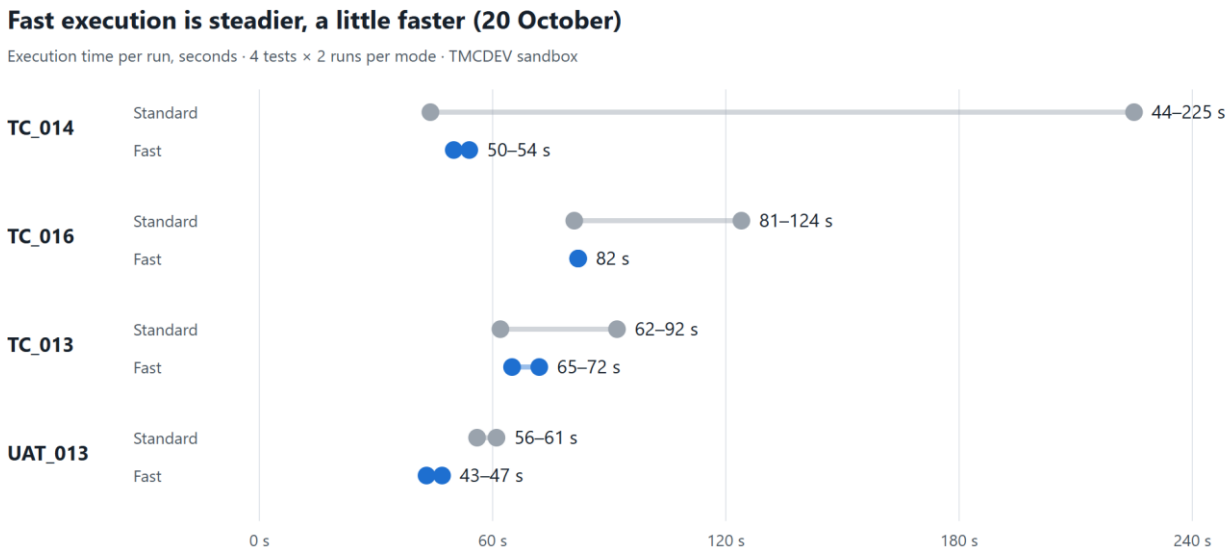


Figure 14. Execution time per run on 20 October, standard against Fast, two runs of each of four test cases on the TMCDEV sandbox. Verdicts, steps and evidence were the same in both modes.

Measurement	Result
Typical gain	About 1–12% per test case. Fast is mainly steadier: standard runs of TC_014 took 44–225 s, Fast runs 50–54 s.
Where the time goes	About 85% of a run is Claude reading Salesforce pages and deciding what to do; login and reporting are small.
Batch baseline (21 Oct)	Five test cases as one batch with Fast on: 300 s and 295 s (two runs), about 1 minute per test, all evidence complete and the same verdicts both times.
Goal	5 test cases within 150 s on re-runs, by replaying steps already proven in earlier runs (in progress, section 18).

> **Honest reading.** The speed-up so far is modest. The larger saving needs Claude to spend less time discovering pages it has seen before, which is the next step.

12. Projects and org folders

The **Project** section of the panel keeps each project's results apart, per Salesforce org. All folder work is done by the local agent server; the extension cannot write files itself.

Folder	Contents
QA-Workspace/Projects/<project>/	project.json: the project's orgs and their folder names.
…/<org>/Test Execution Reports/	Test_Execution_Report.html, execution-data.js, one RUN-… folder per run with its screenshots and execution.json.
…/<org>/Bug Reports/	Bug_Report.html, bug-data.js, one BUG-… folder of evidence per bug. Bug numbers start at BUG-001 per project and org.
…/<org>/Notes/	One .md file per note; list, create, edit and delete from the panel.
…/<org>/Test Cases/	Test cases generated from Jira stories and their Excel exports.
…/<org>/Test Sets/	Saved Sanity and Regression sets with their run history.

- **+ New project** creates the project. Selecting it adds a folder for the selected org, named after its environment (DRIVE-ENERGY-4828 - TMCDEV becomes TMCDEV).
- **Test Execution Reports / Bug Reports / Notes / Open folder** open that project's folder for the org, so TMCDEV shows only TMCDEV runs.
- **No project (shared reports)** works as before, in qa/. Runs recorded before projects existed stay in qa/ unchanged; nothing is moved or deleted.
- `QA_WORKSPACE_DIR` in .env moves the workspace, for example to D:\QA-Workspace. Names are restricted to letters, digits, spaces and - _ . & ( ) ' +.

13. Jira story to test cases

The **Jira story** tab turns a pasted story into test cases. Generation takes about a minute. Nothing runs in Salesforce until the tester starts the approved cases.

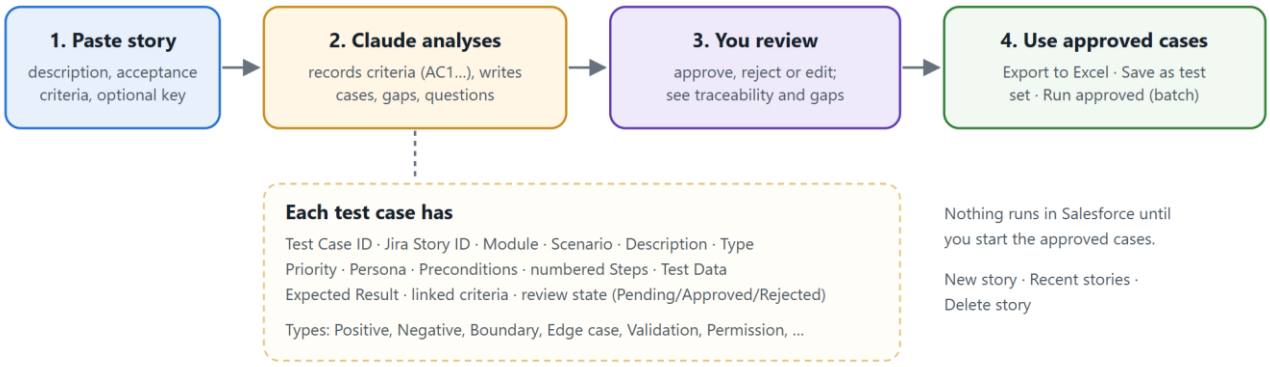


Figure 15. From story to approved test cases. Claude records the acceptance criteria first, then writes cases linked to them; the tester decides which cases are used.

1. Paste the story (summary, description, acceptance criteria, rules, notes) and optionally its Jira key, then click **Generate test cases**.
2. Claude lists the criteria and writes cases where they apply: positive, negative, boundary and edge, validation, permission and persona, data integrity, integration and regression. Negative cases are favoured and duplicates dropped.
3. Review each case: **Approve**, **Reject** or **Edit** any field. The coverage panel shows traceability, criteria without a case, gaps, assumptions, questions for the business and possibly affected areas.
4. **Export all / Export approved** to Excel (Test Cases, Traceability and Notes tabs, plus empty Actual Result, Status and Bug ID columns). The file can be uploaded again in Upload file.
5. **Run approved (N)** runs the approved cases as a batch; **Save approved as test set** keeps them as a Sanity or Regression set.

New story starts over, **Recent stories** reopens an earlier story of the org, and **Delete story** removes its saved cases (exported Excel files are kept). Cases are saved in <project>/<org>/Test Cases/, or qa/test-cases/ without a project.

14. Sanity and Regression test sets

The **Test sets** tab keeps named sets of test cases per org and project, run with one click. **Sanity** is a quick check of the main flows; **Regression** is the full set, re-run after every change.

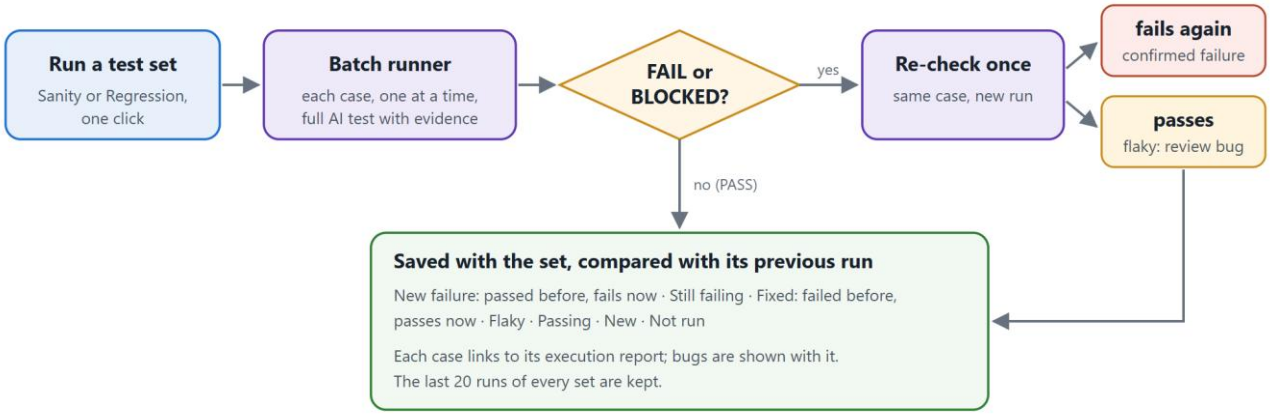


Figure 16. Running a test set. Each FAIL or BLOCKED case is re-checked once; the final results are saved with the set and compared with its previous run.

Result	Meaning
New failure	Passed in the previous run, fails now. Listed first: the most likely regression.
Still failing	Failed in both runs.
Fixed	Failed before, passes now.
Flaky	Failed, then passed on the re-check. Its bug from the first attempt stays for review.
Passing	Passed in both runs.
New / Not run	Not in the previous run / not run this time (for example stopped).

- **Create** a set from Upload file (**Save as test set** for the selected rows) or from the Jira story tab (**Save approved as test set**). Names are unique per org and project.
- **Run** uses the selected persona, project and Fast execution switch; progress shows in Upload file. **Re-check failures once** is on by default for sets.
- Each case links to its execution report. The last 20 runs of a set are kept; deleting a set keeps its runs' reports and screenshots.
- Sets are saved in <project>/<org>/Test Sets/, or qa/test-sets/<org>/ without a project.

15. Security and safeguards

Area	Safeguard
Network exposure	The server listens on 127.0.0.1 only and rejects requests from other hosts and from web pages' origins; it cannot be reached from the network.
Who can call the server	Every API call needs the agent token (compared in constant time). The token lives in the extension's background worker, never in the Salesforce page. Reports use a separate unguessable link key derived from the token.
Claude's tools	Claude Code runs with no built-in tools (no shell, no file reads or edits). Test runs get only the browser and QA tools; reviews get only the review tools.
Tool-call isolation	Each run or review gets its own random key, so a Claude process can only report into its own run. Every tool call is schema-validated.
Salesforce credentials	No persona passwords exist. The admin session is saved locally under `auth/`; the persona session file is deleted after every run, and any left over from an interrupted run is removed at start-up.
Your own browser	Tests run in a separate Playwright browser; the tester's Chrome session is never used or changed.
Test data	Claude may read with single, bounded SOQL SELECT queries; may create only business records (standard business objects and custom objects); may edit only records created for testing; may not delete anything or change Setup.
Untrusted content	Test cases and BRDs are passed to Claude as data, with instructions to ignore any instructions inside them; panel text is rendered without HTML, so pasted text cannot inject markup.
AI usage and cost	Claude runs on the signed-in Claude Code seat (Sonnet by default); API keys are deliberately not passed to it. Runs have a turn limit and a 30-minute timeout.
Files and folders	Project, org and note names are checked against an allowed character set, and every path is checked to stay inside the workspace, so a name cannot reach other folders.
Jira keys	The story key is cleaned (letters, digits, - and _ only, at most 40 characters) before it is used in file names.
API checks	Only GET on the org data API; writes only to Apex REST or to external systems marked allowWrite. Secrets are injected by the server and never shown to Claude.
Fast execution	Claude cannot use the browser until the persona login is confirmed; a failed login still blocks the run.

16. Operating the agent

Daily use

1. Start the server in a terminal and keep it open: `npm start`
2. Open any Salesforce page and click the shield button (bottom right).
3. Check the org shows "connected"; click **Reconnect** if the admin session has expired.
4. Pick the persona, then paste a test case and click **Run test**, or upload a sheet in **Upload file**.
5. Optionally add the BRD and click **Analyse** before running the batch.
6. Open the reports from the panel when the run or review finishes.

Commands

Command	Purpose
`npm install`	Install dependencies (first time).
`npm start`	Start the agent server.
`npm run build:extension`	Build the extension into `packages/extension/dist`; then reload it in `chrome://extensions`.
`npm run connect -- --org <id>`	Connect an org from the terminal.
`npm run connections -- --verify`	List orgs, admin connections and personas, and check each session still works.
`npm run check`	Typecheck, lint and the 197 automated tests.

Settings (.env)

Setting	Meaning
`AGENT_TOKEN`	Shared secret pasted into the extension settings (at least 16 characters).
`PORT`	Server port, default 4317.
`BROWSER_CHANNEL`	`chrome` (installed Chrome) or `chromium`.
`HEADLESS`	`false` to watch test runs in a visible window.
`CLAUDE_MODEL`, `CLAUDE_MAX_TURNS`, `RUN_TIMEOUT_MINUTES`	Optional: model (default sonnet), turn limit (150) and per-run timeout (30 minutes).
HYBRID_EXECUTION	false to use one Claude decision per browser action (default true).
FAST_EXECUTION	true makes Fast execution the default for runs that don't set the switch.
QA_WORKSPACE_DIR	Where the project workspace lives (default QA-Workspace in the project folder).

Where things are kept

Location	Contents
`config/orgs.json`	Detected orgs and their personas.

Location	Contents
`auth/<org>/`	Saved admin session per org (sensitive: never share).
`qa/`	Reports, data files, screenshots, bug evidence, test-data registry.
`runs/`	Raw record of each run (run.json).
QA-Workspace/	Projects, their org folders (reports, bugs, notes, test cases, test sets) and shared report assets.
qa/test-cases/, qa/test-sets/	Jira test cases and test sets created without a project.
config/apis.json	External systems Claude may call in API checks (optional).

17. Change history (6–21 October)

What was added after the first version of this report, in order. Each change kept the earlier behaviour available: new options are switches, and older data is never moved.

Date	Version	What changed
9 Oct	12265bb	Hybrid execution (run_steps, 21 generic actions, Salesforce-aware waits, fallback to single tools); run metrics; multi-persona tests; API checks with config/apis.json; requirements review in the panel.
19 Oct	e22257a	Run results and report links stay in the panel when Chrome suspends the extension.
20 Oct	caa9d95	Projects and org folders in QA-Workspace with notes; New test case button.
21 Oct	c82e69f	Jira story to test cases; Fast execution switch; execution time in the reports; fixes to generation and review.
21 Oct	not yet committed	Sanity and Regression test sets, re-check of failures, regression comparison; Jira New story, Recent stories and Delete story.
In progress	–	Replay of steps proven in earlier runs (Fast execution only). Baseline measured: 295–300 s for five test cases.

Problems found and fixed on the way

- Two Jira generations started at the same moment could overlap, and two in the same second could overwrite each other: fixed.
- Edit on a collapsed Jira test case did nothing: it now opens the case.
- Finished test set runs could be shown as interrupted: fixed.
- Turning the Fast switch on and off made the panel jump: fixed.
- A Jira key with unsafe characters could reach a file name: now cleaned.

18. Limitations and next steps

Current limitations

- **Runs on one machine.** The server runs on the tester's PC and must stay running in a terminal; there is no shared or hosted server yet.
- **One test at a time.** Runs are sequential by design (shared admin session); a large batch takes as long as its tests combined.
- **One review at a time;** up to 200 test cases and about 400,000 characters of BRD text per review. Scanned (image-only) PDFs cannot be read; Word .doc and Excel .xls must be re-saved as .docx / .xlsx.
- **AI judgement.** Claude's step results and review verdicts are evidence-backed but should be spot-checked; reviewers can change any status in the reports with a note.
- **Reports are local files.** They are shared by link only while the server runs, or by printing to PDF / exporting CSV.
- **Data quality.** Module names come from the test sheets, so variants such as "Account" and "Accounts" appear separately until the sheets are made consistent.
- **Login expiry.** Long batches depend on the admin session; when Salesforce expires it, the remaining runs are BLOCKED until Reconnect.
- **Flaky is a hint, not a verdict.** A case that passes on its re-check still needs a look at its first failure.
- **Jira is pasted, not read.** Reading stories directly needs the Atlassian connector to be authorised.
- **Speed.** Most of each run is still Claude discovering pages; the proven-steps replay is not finished yet.

Suggested next steps

1. Run the agent server as a background Windows service so it does not depend on an open terminal.
2. Use the requirements review on each new sprint's test sheet before execution, and add the suggested missing test cases.
3. Before real testing starts, archive the trial runs and bugs so the reports start empty, then agree a triage routine for the real defects as they are found.
4. Standardise module names and persona labels in the test sheets.
5. Consider a shared, access-controlled location for the reports so management can open them without the tester's machine.
6. Finish the proven-steps replay for Fast execution and measure it against the 295–300 s baseline.
7. Keep the admin session alive during long batches, so they are not blocked half-way.
8. Run a Regression set after every deployment and triage New failures first.
9. Run tests in parallel once sessions can be kept apart safely.

> **Note on the README.** The project README still describes the agent as "Phase 1 (walking skeleton)" and says the server holds a Claude API key. Both are out of date: AI execution is in place and Claude runs on the Claude Code seat. Updating that section is a small follow-up.